University of Westminster
Department of Computer Science

7BUIS027C.2 – Cloud Computing Applications 2026	
Module leader	Mr. Indrajith Ekanayake
Unit	Coursework
Weighting:	50%
Qualifying mark	40%
Description	Report and viva
Learning Outcomes Covered in this Assignment:	LO1, LO2, LO3, LO4, LO5
Handed Out:	9th of Feb 2026
Due Date	16th of April 2026, 23.59 pm IST
Expected deliverables	A **PDF** file explaining the proposed solution for the given problems. The file may contain architecture diagrams, screenshots, pseudocode, etc. with explanations.
Method of Submission:	Online via Blackboard. Attending CW VIVA is compulsory. Failing to do so will set you CW grade to a "Fail."
Type of Feedback and Due Date:	Written feedback and marks 8 weeks after the submission deadline. All marks will remain provisional until formally agreed upon by an Assessment Board.

Copying and plagiarism

Any external sources utilized should be correctly referenced using a common referencing technique (e.g., the Harvard technique). For more details on referencing please visit https://www.westminster.ac.uk/current-students/studies/study-skills-and-training/research-skills/referencing-your-work.

Copying and plagiarism carry severe penalties. Please note that the University offers an online learning tutorial designed to help students understand and avoid plagiarism. This can be accessed by any student under My Organisation on Blackboard. The tab is labelled 'Avoiding Plagiarism'.

Penalty for Late Submission

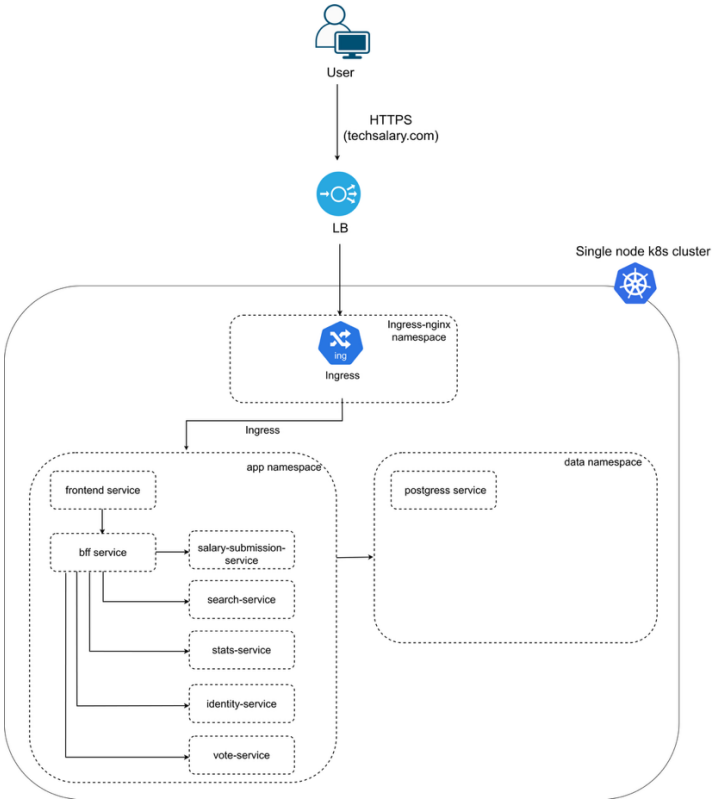
If you submit your coursework late but within 24 hours or one working day of the specified deadline, 10 marks will be deducted from the final mark, to minimum of the pass mark (50%), as a penalty for late submission. If you

submit your coursework more than 24 hours or more than one working day after the specified deadline you will be given a mark of zero for the work in question unless a claim of Mitigating Circumstances has been submitted and accepted as valid.

It is recognised that on occasion, illness or a personal crisis can mean that you fail to submit a piece of work on time. In such cases you must inform the Faculty Registry Office in writing on a mitigating circumstances form, giving the reason for your late or non-submission. You must provide relevant documentary evidence with the form. This information will be reported to the relevant Assessment Board that will decide whether the mark of zero shall stand. For more detailed information regarding University Assessment Regulations, please refer to the following website: http://www.westminster.ac.uk/study/current-students/resources/academic-regulations.

Coursework Description

You are about to create, build, and deploy a microservice application for tech salary transparency in Sri Lanka. First, refer to the following microservice architecture.



This system is a community-driven technology salary transparency website similar to TechPays.com in Europe and https://techsalary.tldr.lk/ in Sri Lanka. Users can anonymously submit salary information, search salaries by country, company, role, and experience level, and vote on whether submitted salaries are trustworthy. User login is required only for community actions such as upvoting/downvoting and reporting entries. To protect privacy, user identities (emails, passwords, account details) are stored separately from salary submissions, and salary records are kept anonymous and are not directly linked to user emails.

The platform is implemented using a microservices architecture, where each major function—salary submission, identity management, voting, search, and statistics—is handled by an independent service. All services are containerized using Docker and deployed within a single-node Kubernetes cluster, which provides consistent deployment, isolation between services, and easier management.

External traffic is routed through a load balancer and Kubernetes Ingress, which directs requests to the correct services inside the cluster. Application microservices run in the app namespace, while PostgreSQL runs in the data namespace to isolate persistent storage from application workloads.

Explanation of each service in the architecture

- **Frontend service:** The frontend service provides the user interface of the website. It displays salary search pages, submission forms, statistics pages, and login screens (only for voting people need to log in). All API requests from the browser are sent to the BFF service rather than directly to backend microservices.
- **BFF service (Backend-for-Frontend):** The BFF service acts as the single entry point for the frontend. It handles request routing, authentication checks, caching, and communication with internal microservices. This keeps the frontend simple and prevents direct exposure of internal services.
- **Salary-submission service:** It validates the data, applies anonymization rules, and stores the raw submission in the database. No personal identity information such as email is stored here and users do not need to login to submit salary.
- **Identity service:** The identity service manages user accounts, login, and authentication tokens. User emails and passwords are stored separately from salary data to protect privacy. Other services only receive a user ID, never personal details.

- **Vote service:** The vote service allows logged-in community members to upvote or downvote salary submissions. When a submission reaches a predefined voting threshold, it is marked as approved.
- **Search service:** The search service provides a filtered salary lookup based on country, company, role, level, and other attributes.
- **Stats service:** The stats service calculates and returns aggregated salary insights, such as averages, medians, and percentiles for specific locations or roles.
- **PostgreSQL service (data namespace):** Single database instance, logically separated using schemas: 1)identity schema for user accounts, 2) salary schema for submissions and approved records, 3) community schema for votes/reports, 4)Uses persistent storage so data survives restarts.

Implement the complete system described above and deploy it to a single-node Kubernetes cluster on Azure. Your submission must include working containers, Kubernetes manifests, PostgreSQL schema setup, and proof of the full workflow (submission → voting → approval → search → stats). Provide screenshots and/or terminal outputs that demonstrate each step.

Coursework Marking scheme

The Coursework will be marked based on the following marking criteria:

Criteria	Mark	
Students clearly describe the goal of the system and the required workflow. The report must explain that salary submission is anonymous (no login), login is required only for community actions (upvote/downvote/report), and privacy is protected by separating identity data from salary submission data. The explanation must match the required workflow submission → voting → approval → search → stats.	10 Marks	
Students correctly implement the services described in the architecture with clear responsibilities and correct interactions. The frontend must provide the UI and call only the BFF. The BFF must act as the single entry point and enforce authentication for voting-related actions. The salary-submission service must validate input, store submissions as PENDING, and store the anonymize flag. The identity service must implement signup/login and token generation. The vote service must record votes and apply an approval threshold to mark submissions as APPROVED.	20 Marks	
Students demonstrate correct cloud-native deployment on a single-node Kubernetes cluster. Each service must be containerized using Docker and deployed as a separate Kubernetes Deployment with a corresponding ClusterIP Service. The solution must correctly use namespaces: all microservices must run in the app namespace, while PostgreSQL runs in the data namespace. External traffic must be routed through a load balancer and Kubernetes Ingress, and the ingress rules must correctly forward requests to the intended entry point (frontend/BFF). Configuration must be externalized using ConfigMaps and Secrets, services should remain stateless, and basic readiness/liveness behaviour should be shown (for example using probes or restart evidence).	30 Marks	
Students set up a single PostgreSQL instance with correct logical separation using schemas. The identity schema must contain user and authentication tables. The salary schema must include raw submissions (PENDING), approved submissions (APPROVED), and any required aggregate tables. The community schema must include votes (and reports if implemented). Students must also demonstrate that persistent storage is enabled (PVC), and that identity information such as email is not stored in salary submission tables.	15 Marks	
Students implement privacy and security requirements correctly. Passwords must be stored securely (hashed, not plain text), authentication must be enforced for voting/reporting endpoints, and internal services must not be directly exposed publicly (only accessible through ingress/BFF). The anonymize toggle must be implemented so that public records hide or generalize identifying information when enabled. The design must ensure salary records are not directly linked to user emails.	10 Marks	
Students provide screenshots and/or terminal outputs proving the system works end-to-end. Evidence must show a salary submission being stored as PENDING, a user successfully signing up/logging in and receiving a token/session, a vote being recorded, the submission becoming APPROVED once the threshold is reached, the approved salary appearing in search results, and the stats output updating based on approved	10 Marks	

salaries.		
Students submit complete and reproducible Kubernetes manifests including namespaces, deployments, services, ingress, and persistent volume resources for PostgreSQL. The repository must be well structured and include a README with exact commands to build container images, apply manifests, initialize the database schema, and test the workflow. Configuration and secrets must not be hardcoded in source code.	5 marks	
TOTAL	**100**	